SDF Final Project

Bokinela Praneeth Kumar
cs24btech11013

May 2025

Contents

Chapter 1

Introduction

In this project we built a library, arbitrary-precision arithmetic is crucial for the applications involving caluclations beyond the range of built-in data types. This project implements a library in Java, which supports addition, subracting, multiplication and division of both arbitrarily large integers and floating-point numbers. This report contains informartion about the project. It includes the project design, implementation, class structure, etc.

Chapter 2

Project Design

2.1 Class Structure

This project is organized into the following classes:

- **AInteger**: Handles arbitrary-precision integer operations.
- **AFloat**: Handles arbitrary-precision floating-point operations.
- **MyInfArith**: Provides the command-line interface.

2.2 UML Class Diagram

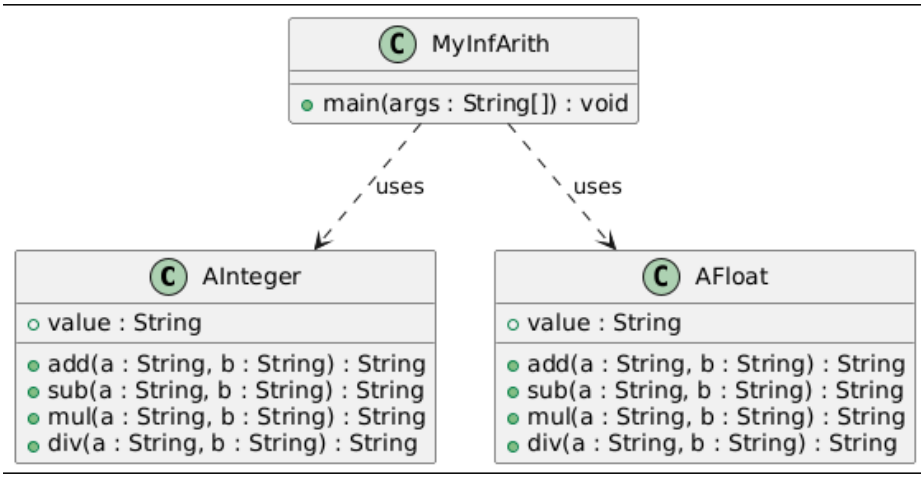

Figure 2.1: UML Class Diagram of the Project

2.3 Design Decisions

- Arbitrary length numbers are stored as strings.
- Seperated integer and float logic for maintainability.
- Ant and python scripts for building and running the project.

2.4 Directory Structure

- All compiled `.class` files are placed in a dedicated `build/` directory inside the project directory, separate from the source code.
- The `arbitraryarithmetic/` directory (inside the project directory) contains `AInteger.java`, `AFloat.java`, and the generated `aarithmetic.jar` file.
- Along with `build/` and `arbitraryarithmetic/`, the project directory also contains `build.xml`, `MyInfArith.java`, and `myScript.py`.

Chapter 3

Implementation

3.1 Code Organization

- `arbitraryarithmetic/AInteger.java`: Integer operations.
- `arbitraryarithmetic/AFloat.java`: Floating-point operations.
- `MyInfArith.java`: Main class with `main()` method.
- `build.xml`: Ant build file.
- `myScript.py`: Python script for building/running.

3.2 Functionality

3.2.1 AInteger Class

- The methods `addString`, `subString`, `mulString`, and `divString` perform basic addition, subtraction, multiplication, and division on numeric strings. These methods focus on the core arithmetic logic and do not handle all edge cases, such as differing signs or invalid inputs.
- The methods `add`, `sub`, `mul`, and `div` serve as wrappers that handle input validation and all cases involving different signs. They ensure that the final result of the operation is correct, even for negative numbers or invalid input formats.

3.2.2 AFloat Class

- The methods `add`, `sub`, `mul`, and `div` in the `AFloat` class implement floating-point arithmetic by internally leveraging the string-based arithmetic methods (`addString`, `subString`, `mulString`, `divString`) from the `AInteger` class.
- For each operation, the methods first manipulate the positions of the decimal point ('.') in the input strings.
- They remove the decimal points to convert the floating-point numbers into integers, perform the arithmetic using the corresponding `AInteger` method, and then re-insert the decimal point at the correct position in the result to produce the final floating-point output.

Chapter 4

README

You can build and run this project on your machine via two approaches : Ant
Python-Script

4.1 Ant Approach

- Run `Ant jar` on your command line.
- To divide 3227.0 555.0, run:

  ```
  java -cp arbitraryarithmetic/aarithmetic.jar MyInfArith float div 3227 555
  ```

- Replace `float div 3227 555` with your desired operation and operands.

4.2 Python Script approach

- To div 3227 555, run:

  ```
  python3 myScript.py float div 3227 555
  ```

- Replace `float div 3227 555` with your desired operation and operands.

Chapter 5

Verification

Few tested cases and their results:
Input:
`java MyInfArith int add 2365007822491294949731093324025042939783262467113798386384401498`
Output:
`665898614873800632956973176417485774524230034638114444645388400`
Output:
`-546287306263397813379509411254315774524230034638114444645388400`
Output:
`java MyInfArith int mul 1434416316044592994268069731232223017167694823904478474013730519`
Output:
`330162008905899217578310782382075660760972861550182008086155118`
Output:
`java MyInfArith int div 879272636528306057983395052167721149383525361708964745499835817804979`
Output:
`java MyInfArith float div 879272636528306057983395052167721149383525361708964745499835817804979.0`
Output:
`17804979.091469989302961159520087878533615460923.263771`
Output:
`java MyInfArith float sub 840196454.51725 712586963.70283`
Output:
`127609490.81442`
Output:
`java MyInfArith float mul 6400251.9377695 2326541.6827934`
Output:
`14890452913599.97174572532130`

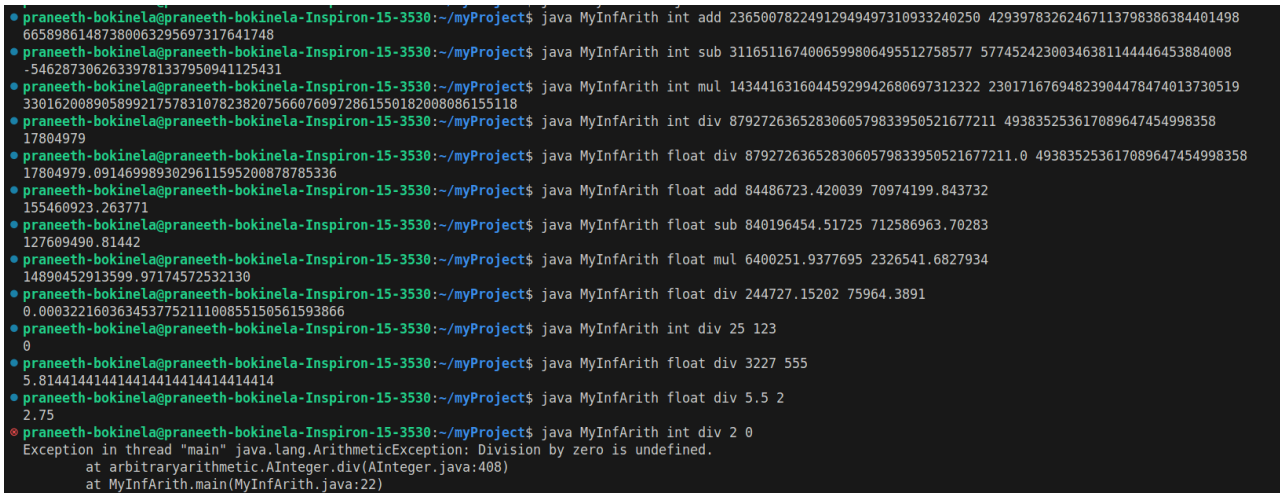

Figure 5.1: Snapshot of testing

Chapter 6

Limitations

- Floating-point division sometimes places the decimal incorrectly for certain cases (e.g., 1.25/0.05 gives 2500 instead of 25).
- Not all decimal fractions can be represented exactly, so rounding errors may occur.

Chapter 7

Key Learnings

- Gained practical experience with Object-Oriented Programming (OOP).
- Learned to set up and automate the build process using Ant.
- Understood version control systems and it's ability to track changes.
- Got familiar with basic Git workflow.
- Acquired the basics of technical documentation and report writing using LaTeX.

Chapter 8

Git Snapshot

```
Author: Praneeth <cs24btech11013@iith.ac.in>
Date:   Fri May 2 14:50:22 2025 +0530

    Fixed a bug in build.xml

commit c7793844f426764d97360194177044f63600a2fd
Author: Praneeth <cs24btech11013@iith.ac.in>
Date:   Fri May 2 10:41:15 2025 +0530

    Added comments in AFloat.java

commit 8431211fcd324a29fae3428c3d6b1de1fc00fc5e
Author: Praneeth <cs24btech11013@iith.ac.in>
Date:   Fri May 2 10:20:01 2025 +0530

    Fixed typos in AInteger.java

commit f89e5ce976f0585391b56d7b4832d54181877845
Author: Praneeth <cs24btech11013@iith.ac.in>
Date:   Fri May 2 10:15:45 2025 +0530

    Added comments to AInteger.java

commit 505ebed59de6e5ecb7958ffa9628821a441c0334
Author: Praneeth <cs24btech11013@iith.ac.in>
Date:   Fri May 2 04:31:59 2025 +0530

    Added .gitignore MyInfArith makeFile python Script.

commit 4a99b51ff20016ca55e3ace5a7140b93b4630dae (tag: aint,afloat-complete)
Author: Praneeth <cs24btech11013@iith.ac.in>
Date:   Fri May 2 04:19:27 2025 +0530

    Implemented helper functions in AFloat

commit 0f3060a325e9f8f99820bda2fcffadd5d3dddbd9
Author: Praneeth <cs24btech11013@iith.ac.in>
Date:   Fri May 2 03:24:53 2025 +0530

    Implemented div method using divString in AInteger and fixed some bugs

commit 310d8762e68e783cb26df0e82a6de9f85a48a53b
Author: Praneeth <cs24btech11013@iith.ac.in>
Date:   Fri May 2 03:23:29 2025 +0530
```

Figure 8.1: Git snapshot